

A Service Envelope System for Orchestrating Requests: Future-Proofing MAPUA's School Information System (SIS) Service Center

Abstract: This research paper proposes a novel Service Envelope system as a foundational architecture for managing and processing diverse user requests within a custom School Information System (SIS). The system leverages a sequential, state-based processing model encapsulated within distinct "Envelopes" (Request, Approval, Payment, Processing, Delivery, Feedback). Central to this proposal is the use of YAML as a flexible, human-readable, and machine-interpretable language for defining and orchestrating service requests. By using YAML for each Envelope, the system achieves high generalizability, allowing for the creation of multiple service types from a common template. This paper outlines the architectural solution, details the proposed YAML schema for each Envelope, and discusses implementation considerations for building a transparent and efficient SIS.

Keywords: Service Envelopes, Request Processing, SIS, School Information System, Workflow Orchestration, YAML, State Machine, Business Process Automation, User Experience.

1. Introduction

Modern School Information Systems (SIS) are complex ecosystems that manage a myriad of interconnected processes, ranging from student admissions and course enrollment to grade reporting, financial aid, and transcript requests. Each of these "services" involves distinct steps, approvals, data inputs, and outputs, often leading to a fragmented and opaque user experience. The challenge lies in building a robust, flexible, and transparent framework capable of orchestrating these diverse requests in a standardized yet adaptable manner.

This paper introduces the "Service Envelope System," a structured approach to building the foundation for processing user requests in a custom SIS. Inspired by the concept of a physical mail Envelope that contains specific items for a journey, the Service Envelope encapsulates the entire lifecycle of a request through a series of sequential, logical stages. The core innovation lies in leveraging YAML (YAML Ain't Markup Language) to define the structure, parameters, and state transitions within each Envelope, enabling a declarative, generalized, and highly customizable workflow. This approach aims to provide a clear processing flow, improve system transparency, and enhance the user's ability to track the status of their requests.

2. Background: Challenges in SIS Request Management

Traditional SIS architectures often rely on tightly coupled modules or ad-hoc scripts to handle specific request types. This leads to several common challenges:

- **Lack of Standardization:** Different request types follow different, often inconsistent, processing paths.
- **Opaque State Transitions:** Users struggle to ascertain the current status of their request, leading to frequent inquiries and frustration.
- **Limited Transparency for Administrators:** Tracking the progress of multiple requests and identifying bottlenecks is difficult.
- **Rigidity and Maintenance Burden:** Modifying or adding new service types requires significant code changes and extensive testing.
- **Poor Auditability:** Tracing the history and actions taken on a request can be cumbersome.
- **Inconsistent User Experience:** Different services present different interaction patterns, leading to a steep learning curve.

The Service Envelope system seeks to address these challenges by providing a generalized, state-based, and human-readable framework for request orchestration.

3. The Service Envelope System: Core Concept

The Service Envelope system defines a hermetic, sequential processing pipeline for every user request. Each request is instantiated as a "Service Processing Envelope" that traverses through a predefined series of encapsulated stages called "Envelopes." The system's core principle is that a request only moves to the next Envelope once the preceding one has satisfied its conditions or reached a terminal state (completion, rejection, or waiver).

This structured approach offers:

- **Modularity:** Each Envelope focuses on a specific aspect of the request lifecycle, promoting clean separation of concerns.
- **Transparency:** The sequential nature and defined states within each Envelope make the request's progress easily understandable.
- **Auditability:** Every transition and state change can be meticulously logged, providing a clear audit trail.
- **Flexibility & Generalization:** By defining the Envelope structure and processing rules in a declarative language like YAML, the system can adapt to a wide variety of service types without requiring code changes for each new service.
- **Enhanced User Experience:** A clear "visual queue" in the UI allows users to intuitively track their requests, reducing uncertainty and support overhead.

4. Envelopes in Detail: The Request Processing Structure

The Service Envelope system is composed of the following sequential Envelopes, as defined in [Project SIS Documentation \(Provided by Sir Danny\)](#) :

- **Request Envelope:** The initiation point. Contains all static information about the request, its unique identifier, initiator, timestamp, and core parameters specific to the service being requested.
- **Approval Envelope:** Handles necessary authorizations. This Envelope determines if any approvals are required, identifies the approvers, and tracks the approval status. It can be bypassed if no approvals are needed.

- **Payment Envelope:** Manages financial settlements. This Envelope becomes active if the service incurs a fee. It tracks payment status, amount, and transaction details. It can also be bypassed if the service is free.
- **Processing Envelope:** Orchestrates the core backend logic. This is where the actual work of fulfilling the request (e.g., student registration, transcript generation) takes place. It tracks the progress of internal tasks.
- **Delivery Envelope:** Specifies how the service output is conveyed. This Envelope defines the method and details for delivering the result of the request (e.g., email, physical mail, digital download).
- **Feedback Envelope:** Provides a mechanism for user feedback. This post-completion Envelope allows for collecting user satisfaction data, linking to surveys, or providing follow-up information. It is often optional or triggers after delivery.

The system's state transitions are governed by rules defined within or referenced by the YAML configuration. For instance, if the Approval Envelope's status is "Rejected," the entire request's `overallStatus` transitions to "Failed" or "Rejected," and subsequent Envelopes are bypassed.

5. YAML as the Orchestration Language

YAML's intrinsic qualities make it an ideal choice for defining the Service Envelope system's configurations:

- **Human Readability:** Its clean syntax with indentation makes it easy for developers, administrators, and even non-technical stakeholders to understand the structure of a request.
- **Hierarchical Structure:** Naturally maps to the nested nature of Envelopes and their associated parameters.
- **Versatility:** Can represent complex data structures, lists, and key-value pairs, accommodating diverse service requirements.
- **Version Control Friendliness:** Being plain text, YAML files integrate seamlessly with version control systems (e.g., Git), allowing for tracked changes to service definitions.
- **Declarative Nature:** Defines *what* needs to be done, rather than *how*, allowing the underlying orchestrator to handle the procedural logic.

The core idea is that a single YAML document, or a collection of linked YAML fragments, will define the entire lifecycle of a specific service request. This YAML is then interpreted by a central "Service Envelope Orchestrator" responsible for managing state transitions, invoking external services, and updating the request's status.

6. Proposed Architecture and Solution Design

The Service Envelope system would be implemented as a distributed or modular architecture:

1. **Request Ingress:** An API endpoint or message queue receives new service requests from the user interface or other systems.
2. **Service Envelope Orchestrator:** This core component is responsible for:
 - Instantiating a new Service Envelope based on the incoming request and the predefined YAML service template.

- Parsing and validating the YAML configuration.
 - Managing the state of the request across Envelopes.
 - Invoking "Envelope Processors" for each active Envelope.
 - Handling transitions between Envelopes based on success/failure/data from the current Envelope.
 - Persisting the current state of all active requests in a durable data store.
 - Notifying the user interface of status changes.
3. **Envelope Processors:** Independent services or modules, each specialized in handling the logic pertinent to a specific Envelope (e.g., an `ApprovalProcessor`, `PaymentProcessor`, `CoreProcessor`). These processors interact with external systems (e.g., payment gateways, student databases, email services).
 4. **Data Store:** A database (SQL or NoSQL) to persist the YAML structure of each active request, including its unique ID, current status, and historical changes.
 5. **User Interface (UI):** A dashboard displaying pending and past requests. Each request shows a visual representation of its progress through the Envelopes (e.g., a progress bar with segmented Envelope icons), and clicking on a request reveals the detailed YAML content (or a human-readable parsed view) for complete transparency.

None

```
graph TD
    A[User/External System] --> B{Request Ingress API};
    B --> C[Service Envelope Orchestrator];
    C -- Retrieves YAML Template --> D[Service Definition Repository (YAML)];
    C -- Creates Request Instance --> E[Request Data Store (Persisted YAML)];
    E -- Triggers --> F[Request Processor];
    F -- Triggers --> G[Approval Processor];
    G -- Triggers --> H[Payment Processor];
    H -- Triggers --> I[Processing Processor];
    I -- Triggers --> J[Delivery Processor];
    J -- Triggers --> K[Feedback Processor];
    K -- Updates Status --> E;
    E -- Notifies --> L[User Interface (Visual Queue)];
    L -- Displays --> A;
```

7. YAML Schema Design for Generalization

The key to generalization lies in a standardized, yet flexible, YAML structure for each service request. Each Envelope will have common metadata (`status`, `timestamp`) and service-specific parameters.

Root Level Structure for a `ServiceRequest`:

None

```
# A single Service Request YAML document
```

```

serviceRequest:
  id: "REQ-SIS-20231026-0001" # Unique identifier for this request
  type: "courseEnrollment" # Defines the specific service this request is
    for
  initiator: "student_id_12345"
  overallStatus: "pending_approval" # Overall status of the entire request
    (e.g., queued, pending_approval, processing, completed, failed,
    cancelled)
  createdAt: "2023-10-26T10:00:00Z"
  lastUpdated: "2023-10-26T10:35:00Z"
  history: # A log of key state transitions for auditing
    - status: "created"
      timestamp: "2023-10-26T10:00:00Z"
      Envelope: "request"
    - status: "submitted_for_approval"
      timestamp: "2023-10-26T10:05:00Z"
      Envelope: "approval"

```

```

Envelopes:
  # Each nested key is an Envelope
  request: # Request Envelope
    # ... schema defined below
  approval: # Approval Envelope
    # ... schema defined below
  payment: # Payment Envelope
    # ... schema defined below
  processing: # Processing Envelope
    # ... schema defined below
  delivery: # Delivery Envelope
    # ... schema defined below
  feedback: # Feedback Envelope
    # ... schema defined below

```

7.1. Request Envelope YAML Schema

Defines the core details and parameters of the request. The `parameters` section is crucial for generalization, as it's a flexible map for service-specific data.

None

```

request:
  status: "completed" # initial: "new", "validated", "completed",
    "failed"
  timestamp: "2023-10-26T10:00:00Z"
  sourceSystem: "student_portal"
  validationStatus: "passed" # e.g., 'passed', 'failed_schema',
    'failed_rules'
  validationErrors: [] # e.g., "- Invalid course code", "- Missing
    student ID"
  parameters: # Service-specific data for the request

```

```
# Example for a 'courseEnrollment' serviceType:
studentId: "STU-XYZ-987"
courseCode: "COMP-101"
semester: "Fall 2024"
preferredSchedule: "morning"
# Example for a 'transcriptRequest' serviceType:
# transcriptType: "official"
# deliveryMethodPreferred: "physical_mail"
# recipientName: "Admissions Office, University A"
# recipientAddress: "123 Main St, Anytown, USA"
# purpose: "university_application"
```

@Baron, we might need to define specific data types for some parameters so that we can control the behaviour of the system. Some data type examples that I was thinking of is as follows;

StudentNo - when this parameter is included, the system should validate the student number in the system in order for the request to proceed successfully.

EnrolledTerm - when this parameter is included, the system should prefill the display field with only the terms in which the student is enrolled (these are for request that we know that they are students).

FutureTerms - when this parameter is included, the system should prefill the display field with only future terms for the specific *Program* or *Program Group* it belongs to.

PastTerms - when this parameter is included, the system should prefill the display field with only past terms for the specific *Program* or *Program Group* it belongs to.

Lets meet up and discuss

7.2. Approval Envelope YAML Schema

Handles the approval process. The `required` flag allows bypassing. `rules` can define complex approval logic.

None

```
approval:
  required: true # Boolean - if this Envelope is needed
  status: "pending_approval" # "pending_approval", "approved",
  "rejected", "waived"
  timestamp: "2023-10-26T10:05:00Z"
  approvers: # Array of potential approvers
    group:
      - id: "MAPUA_MAKATI_JOE_FIELD_20202211"
        role: "Department Head"
```

```

        status: "pending" # "pending", "approved", "rejected"
        comment: ""
        approvedAt: ""
    - id: "MAPUA_MAKATI_JANE_FIELV_2123122"
      role: "Registrar"
      status: "pending"
      comment: ""
      approvedAt: ""
      approvalRules: # e.g., "all_must_approve",
        "any_one_of_role_registrar", "specific_approver_Y"
      type: "all_must_approve"
      notes: "Requires departmental and registrar approval due to course
        overload."

```

7.3. Payment Envelope YAML Schema

Manages payment requirements and status.

None

```

payment:
  required: true # Boolean - if this Envelope is needed
  status: "pending_payment" # "pending_payment", "paid",
    "failed_payment", "waived", "awaiting_confirmation"
  timestamp: "2023-10-26T10:30:00Z"
  amount: 150.00
  currency: "USD"
  paymentMethod: "credit_card" # "credit_card", "bank_transfer",
    "cash_cheque"
  transactionId: "PAY-GATEWAY-TXN-ABC-123" # Populated upon successful
transaction
  paymentGatewayResponse: # Raw response from payment gateway (optional)
    code: "00"
    message: "Successful transaction"
  notes: "Course enrollment fee."
  txn_reference_no: "123121"

```

7.4. Processing Envelope YAML Schema

Details the core backend tasks. Allows for multi-step processing within a single Envelope.

None

```

processing:
  status: "in_progress" # "queued", "in_progress", "completed",
    "failed", "cancelled"
  timestamp: "2023-10-26T10:45:00Z"
  processorId: "backend_worker_01" # Worker or service instance handling
this
  currentTask: "db_update_enrollment"

```

```

    tasks: # An array of discrete tasks for this processing stage
      - name: "validate_prerequisites"
        status: "completed" # "pending", "in_progress", "completed",
"failed"
        notes: "All prerequisites met."
        startedAt: "2023-10-26T10:46:00Z"
        completedAt: "2023-10-26T10:48:00Z"
      - name: "db_update_enrollment"
        status: "in_progress"
        notes: "Attempting to add student to course roster."
        startedAt: "2023-10-26T10:49:00Z"
        errorMessage: ""
      - name: "send_enrollment_confirmation"
        status: "pending"
        notes: ""

```

7.5. Delivery Envelope YAML Schema

Defines how the outcome of the request is delivered.

None

```

delivery:
  required: true # Boolean - if this Envelope is needed
  status: "pending_delivery" # "pending_delivery", "delivered",
"failed_delivery", "not_applicable"
  timestamp: "2023-10-26T11:15:00Z"
  method: "email" # "email", "physical_mail", "sms", "digital_download"
  details: # Method-specific delivery details
    email:
      recipient: "student_id_12345@example.edu"
      subject: "Course Enrollment Confirmation - COMP-101"
      templateId: "enrollment_confirmation_template_v1"
      attachmentUrls: ["https://sis.example.edu/receipts/REQ-0001.pdf"]
    # physical_mail:
    #   address: "Student Hall, Apt 4B, Campus Rd, City"
    #   trackingId: "USPS-XYZ-789"
  deliveryAttempts: 1 # Number of attempts if retries are configured
  lastAttemptAt: "2023-10-26T11:15:00Z"
  notes: "Sending confirmation email."

```

7.6. Feedback Envelope YAML Schema

Provides a mechanism for collecting user feedback.

None

```

feedback:
  required: true # Boolean - if this Envelope is active

```



```
    status: "pending_feedback" # "pending_feedback", "feedback_submitted",
"omitted_by_user"
    timestamp: "2023-10-26T11:20:00Z"
    feedbackLink:
"https://forms.example.edu/survey?reqId=REQ-SIS-20231026-0001"
    submissionDate: "" # Date feedback was submitted
    notes: "Link for course enrollment experience survey."
    auto_close_on_expiry: "2023-11-26T11:20:00Z"
```

Example: Course Enrollment Request YAML (Simplified)

None

```
serviceRequest:
  id: "ENR-SIS-20231026-C010"
  type: "courseEnrollment"
  initiator: "student_id_12345"
  overallStatus: "completed"
  createdAt: "2023-10-26T10:00:00Z"
  lastUpdated: "2023-10-26T11:30:00Z"
  history:
    - status: "created"
      timestamp: "2023-10-26T10:00:00Z"
      Envelope: "request"
    - status: "approved"
      timestamp: "2023-10-26T10:10:00Z"
      Envelope: "approval"
    - status: "paid"
      timestamp: "2023-10-26T10:35:00Z"
      Envelope: "payment"
    - status: "completed"
      timestamp: "2023-10-26T11:10:00Z"
      Envelope: "processing"
    - status: "delivered"
      timestamp: "2023-10-26T11:25:00Z"
      Envelope: "delivery"
    - status: "pending_feedback"
      timestamp: "2023-10-26T11:30:00Z"
      Envelope: "feedback"

Envelopes:
  request:
    status: "completed"
    timestamp: "2023-10-26T10:00:00Z"
    sourceSystem: "student_portal"
    validationStatus: "passed"
    parameters:
      studentId: "STU-XYZ-987"
      courseCode: "COMP-101"
      semester: "Fall 2024"
```

```

approval:
  required: true
  status: "approved"
  timestamp: "2023-10-26T10:10:00Z"
  approvers:
    - id: "faculty_head_john_doe"
      role: "Department Head"
      status: "approved"
      approvedAt: "2023-10-26T10:09:00Z"
  approvalRules:
    type: "all_must_approve"
payment:
  required: true
  status: "paid"
  timestamp: "2023-10-26T10:35:00Z"
  amount: 150.00
  currency: "USD"
  transactionId: "PAY-GATEWAY-TXN-C010"
processing:
  status: "completed"
  timestamp: "2023-10-26T11:10:00Z"
  tasks:
    - name: "validate_prerequisites"
      status: "completed"
    - name: "db_update_enrollment"
      status: "completed"
    - name: "send_enrollment_confirmation"
      status: "completed"
delivery:
  required: true
  status: "delivered"
  timestamp: "2023-10-26T11:25:00Z"
  method: "email"
  details:
    email:
      recipient: "student_id_12345@example.edu"
      subject: "Course Enrollment Confirmation - COMP-101"
feedback:
  required: true
  status: "pending_feedback"
  timestamp: "2023-10-26T11:30:00Z"
  feedbackLink:
    "https://forms.example.edu/survey?reqId=ENR-SIS-20231026-C010"

```

8. Parametric Table for (UI/UX) Field Generation

parameter	field_type	options	version	notes
numCopies	int	{maxLimit: 3}	1.0	Min 1 copy

deliveryMethod	choice	{"email","courier","pickup"}	1.0	Single choice
deliveryMethod_email	choice-trigger	{"email"}	1.0	email

9. Implementation Considerations

- **YAML Parsing & Validation:** Robust libraries are needed to parse YAML and validate it against a predefined schema to ensure data integrity. Custom rules for validation can be built.
- **State Management:** The orchestrator needs reliable persistence for the YAML state of each request. Transactions are crucial to ensure atomicity of state changes.
- **Error Handling and Retries:** Each Envelope processor must handle failures gracefully. The orchestrator should implement retry mechanisms for transient errors and define clear rollback or notification strategies for persistent failures.
- **Security & Authorization:** Access control must be implemented for who can initiate, approve, view, or modify specific requests and their Envelopes.
- **User Interface (Visual Queue):**
 - A dashboard displaying all requests, filterable by status, type, and initiator.
 - For each request, a progress bar or sequential set of icons representing the Envelopes, where completed Envelopes are highlighted and the current active Envelope is distinct.
 - Clicking on a request reveals a detailed view of its YAML content (or a cleanly parsed human-readable representation), including all parameters, statuses within each Envelope, and the history log. This provides the desired "visual queue" and deep transparency.
- **Versioning:** As service definitions evolve, a strategy for versioning YAML schemas is essential to ensure backward compatibility and smooth deployments.
- **Scalability:** The orchestrator and Envelope processors should be designed to scale independently to handle varying loads.

10. Benefits and Challenges

Benefits:

- **Enhanced Transparency:** Users and administrators gain clear visibility into the state and progress of every request.
- **Improved Auditability:** Detailed history logs within the YAML provide a complete audit trail.
- **Increased Flexibility and Extensibility:** New service types can be onboarded rapidly by defining new YAML configurations rather than extensive code changes.
- **Modularity and Reusability:** Envelope processors can be reused across different service types, promoting a microservices-like architecture.

- **Consistent User Experience:** All services, regardless of their complexity, follow a standardized lifecycle, improving usability.
- **Reduced Development Time:** Focus shifts from procedural coding of workflows to declarative configuration.

Challenges:

- **Initial Setup Complexity:** Designing a generalized YAML schema and building a robust orchestrator is a significant upfront effort.
- **Performance Overhead:** Extensive YAML parsing and state updates could introduce latency if not optimized.
- **Debugging Complex Workflows:** While transparent, debugging issues in deeply nested workflows can still be challenging.
- **Data Validation:** Ensuring the integrity and correctness of data submitted via YAML requires sophisticated validation layers.
- **Schema Evolution:** Managing changes to the YAML schema over time requires careful planning and migration strategies.

11. Conclusion

The Service Envelope System, underpinned by a declarative YAML orchestration, offers a powerful and flexible paradigm for managing complex request lifecycles within a custom School Information System. By breaking down service processing into distinct, sequential Envelopes and defining their state through human-readable YAML, the system achieves unprecedented levels of transparency, auditability, and adaptability. While requiring an initial investment in architectural design and implementation, the long-term benefits in terms of development velocity, operational efficiency, and enhanced user experience for SIS cannot be overstated. This approach provides a robust and scalable foundation that transforms the way educational services are defined, managed, and delivered.

12. References

- [YAML Official Specification](#)